The University of Azad Jammu & Kashmir,Muzaffarabad

Department of Software Engineering

DIP Lab

Lab # 02

Task Title: DIGITAL IMAGES, PIXELS, COLOR, AND BASIC TRANSFORMATIONS

Submitted to: Fahad Nisar

Student Name: Feeza Kulsoom

Roll No: 2024-SE-03

Reg Number: 2024-UMDB-004738

1. Objective

Manipulate, inspect, crop, resize, analyze (histograms), and quantize (bit depth) digital images using OpenCV, NumPy, and Matplotlib.

2. Step 1: Virtual Environment Setup

An isolated virtual environment was initialized, libraries installed, and .venv excluded via .gitignore.

```
mkdir dip-lab-02 && cd dip-lab-02
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install numpy opencv-python matplotlib
echo .venv/ > .gitignore
```

```
(.venv) PS C:\Users\feeza\OneDrive\Documents\Desktop\dip-lab-02> pip list
Package         Version
--------------- --------
contourpy       1.4.0
cycler          0.12.1
fonttools       4.67.0
kiwisolver      1.5.1
matplotlib      3.11.2
numpy           2.5.4
opencv-python   5.0.0.93
packaging       26.3
pillow          13.0.0
pip             26.2.1
pyparsing       3.3.3
python-dateutil 3.0.0
six             1.17.0
tzdata          2026.5
```

3. Steps 2 & 3: Image Loading & Inspection

Loaded 'sample.png' (679x580x3, uint8) using cv.imread() with FileNotFoundError validation. Inspected pixel coordinates [0,0] and center [339,290], plus global and channel min/max intensity ranges.

```
# ------------------------------------------------------------
# STEP 2: LOAD IMAGE WITH EXCEPTION HANDLING
# ------------------------------------------------------------
  # Place your own color image (>= 300x300) in the folder and name it 'sample.jpg'
IMAGE_PATH = 'sample.png'


if not os.path.exists(IMAGE_PATH):
    raise FileNotFoundError(
        f"[ERROR] Input image '{IMAGE_PATH}' not found in dip-lab-02 directory."
    )
```

```
img_bgr = cv.imread(IMAGE_PATH)
if img_bgr is None:
    raise FileNotFoundError(
        f"[ERROR] cv.imread failed to load '{IMAGE_PATH}'. Verify file format."
    )
```

```
# ------------------------------------------------------------
# STEP 3: INSPECT THE IMAGE
# ------------------------------------------------------------
height, width, channels = img_bgr.shape
dtype = img_bgr.dtype
```

```
print("\n[STEP 2 & 3: IMAGE INSPECTION]")
print(
    f"  • Dimensions: Height = {height} px, Width = {width} px, Channels ="
    f" {channels}"
```

```
)
print(f"  • Shape Tuple: {img_bgr.shape}")
print(f"  • Data Type: {dtype}")
```

```
# Coordinate indexing: [row, col] -> [y, x]
center_y, center_x = height // 2, width // 2
top_left_pixel = img_bgr[0, 0]
center_pixel = img_bgr[center_y, center_x]
```

```
print(f"  • Top-Left Pixel [0, 0] (BGR): {top_left_pixel}")
print(f"  • Center Pixel [{center_y}, {center_x}] (BGR): {center_pixel}")
```

```
# Dynamic range inspection
print(f"  • Global Minimum Intensity: {np.min(img_bgr)}")
print(f"  • Global Maximum Intensity: {np.max(img_bgr)}")
```

```
ch_names = ["Blue Channel", "Green Channel", "Red Channel"]
for idx, name in enumerate(ch_names):
  c_min = np.min(img_bgr[:, :, idx])
  c_max = np.max(img_bgr[:, :, idx])
  print(f"  • {name} (Index {idx}): Min = {c_min}, Max = {c_max}")
```

```
[STEP 2 & 3: IMAGE INSPECTION]
  • Dimensions: Height = 679 px, Width = 580 px, Channels = 3
  • Shape Tuple: (679, 580, 3)
  • Data Type: uint8
  • Top-Left Pixel [0, 0] (BGR): [ 61 141  96]
  • Center Pixel [339, 290] (BGR): [223 197  86]
  • Global Minimum Intensity: 0
  • Global Maximum Intensity: 255
  • Blue Channel (Index 0): Min = 0, Max = 255
  • Green Channel (Index 1): Min = 0, Max = 255
  • Red Channel (Index 2): Min = 0, Max = 255
```

4. Step 4: Color Conversions & Channel Decomposition

Converted BGR to RGB for accurate rendering in Matplotlib. Decomposed into R, G, B channels and Grayscale. Calculated center pixel RGB [86, 197, 223] and converted to HEX (#56C5DF).

```
# ------------------------------------------------------------
  # STEP 4: CONVERT COLORS AND SEPARATE CHANNELS
  # ------------------------------------------------------------
  print("\n[STEP 4: COLOR CONVERSIONS & CHANNEL DECOMPOSITION]")
  img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)
  img_gray = cv.cvtColor(img_bgr, cv.COLOR_BGR2GRAY)
  b_channel, g_channel, r_channel = cv.split(img_bgr)

  # Center pixel RGB and HEX calculation
  center_rgb = img_rgb[center_y, center_x]
  hex_color = f"#{center_rgb[0]:02X}{center_rgb[1]:02X}{center_rgb[2]:02X}"
  print(f"  • Center Pixel RGB: {center_rgb}")
  print(f"  • Center Pixel HEX Code: {hex_color}")
```

```
  # Display 2x3 Figure
  fig, axes = plt.subplots(2, 3, figsize=(12, 8))
  axes[0, 0].imshow(img_rgb)
  axes[0, 0].set_title("RGB (Correct Display)")
  axes[0, 0].axis("off")
```

```
  axes[0, 1].imshow(r_channel, cmap="gray")
  axes[0, 1].set_title("Red Channel (Grayscale)")
  axes[0, 1].axis("off")
```

```
  axes[0, 2].imshow(g_channel, cmap="gray")
  axes[0, 2].set_title("Green Channel (Grayscale)")
```

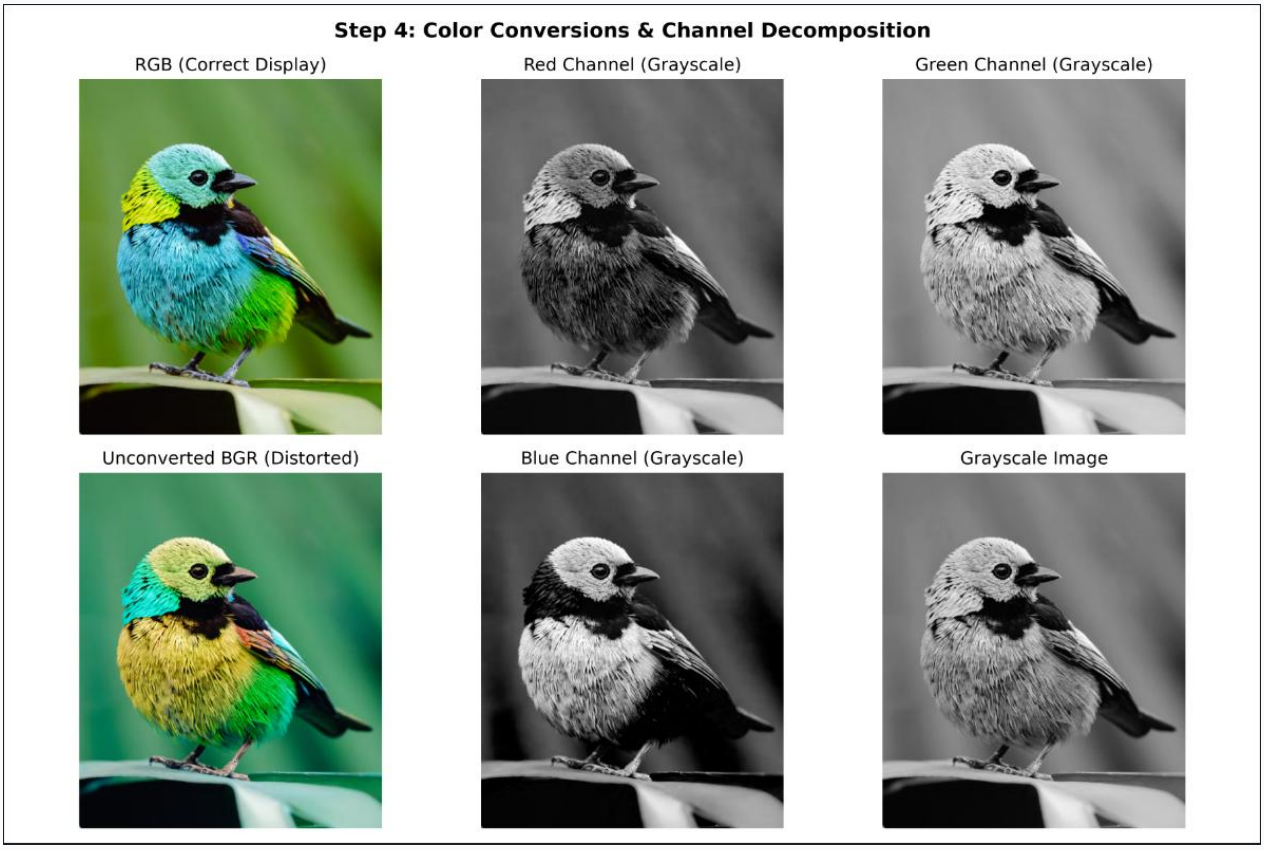
```
axes[0, 2].axis("off")
```

```
axes[1, 0].imshow(img_bgr)
axes[1, 0].set_title("Unconverted BGR (Distorted)")
axes[1, 0].axis("off")
```

```
axes[1, 1].imshow(b_channel, cmap="gray")
axes[1, 1].set_title("Blue Channel (Grayscale)")
axes[1, 1].axis("off")
```

```
axes[1, 2].imshow(img_gray, cmap="gray")
axes[1, 2].set_title("Grayscale Image")
axes[1, 2].axis("off")
```

```
plt.suptitle(
    "Step 4: Color Conversions & Channel Decomposition",
    fontsize=14,
    fontweight="bold",
)
plt.tight_layout()
plt.savefig("channels_comparison.png", dpi=300)
plt.show()
```




```
[STEP 4: COLOR CONVERSIONS & CHANNEL DECOMPOSITION]
  • Center Pixel RGB: [ 86 197 223]
  • Center Pixel HEX Code: #56C5DF
```

5. Step 5: Crop, Resize, and Save

Extracted a centered 100x100 region of interest (Y[289:389], X[240:340]). Resized to 200x200 via bilinear interpolation. Converted back to BGR and saved as 'resized_crop.png'.

```
# ------------------------------------------------------------
# STEP 5: CROP, RESIZE, AND SAVE
# ------------------------------------------------------------
print("\n[STEP 5: CROP, RESIZE & PERSISTENCE]")
crop_size = 100
y_start = max(0, center_y - crop_size // 2)
y_end = y_start + crop_size
x_start = max(0, center_x - crop_size // 2)
x_end = x_start + crop_size

crop_rgb = img_rgb[y_start:y_end, x_start:x_end]
print(f"  • Crop Coordinates: Y[{y_start}:{y_end}], X[{x_start}:{x_end}]")
print(f"  • Crop ROI Shape: {crop_rgb.shape}")
```

```
# Resize crop from 100x100 to 200x200
resized_crop_rgb = cv.resize(
    crop_rgb, (200, 200), interpolation=cv.INTER_LINEAR
)
print(f"  • Resized Crop Shape: {resized_crop_rgb.shape}")
```

```
# 1x3 Side-by-side display
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(img_rgb)
axes[0].set_title(f"Original Image ({width}x{height})")
axes[0].axis("off")
```

```
axes[1].imshow(crop_rgb)
axes[1].set_title("Cropped ROI (100x100)")
```

```
axes[1].axis("off")
```

```
axes[2].imshow(resized_crop_rgb)
axes[2].set_title("Resized Crop (200x200)")
axes[2].axis("off")
```

```
plt.suptitle(
    "Step 5: Spatial Cropping and Bilinear Resizing",
    fontsize=13,
    fontweight="bold",
)
plt.tight_layout()
plt.savefig("crop_and_resize.png", dpi=300)
plt.show()
```

```
# Convert back to BGR for cv.imwrite saving
resized_crop_bgr = cv.cvtColor(resized_crop_rgb, cv.COLOR_RGB2BGR)
save_status = cv.imwrite("resized_crop.png", resized_crop_bgr)
print(f"  • cv.imwrite('resized_crop.png') Success: {save_status}")
```



```
[STEP 5: CROP, RESIZE & PERSISTENCE]
  • Crop Coordinates: Y[289:389], X[240:340]
  • Crop ROI Shape: (100, 100, 3)
  • Resized Crop Shape: (200, 200, 3)
  • cv.imwrite('resized_crop.png') Success: True
```

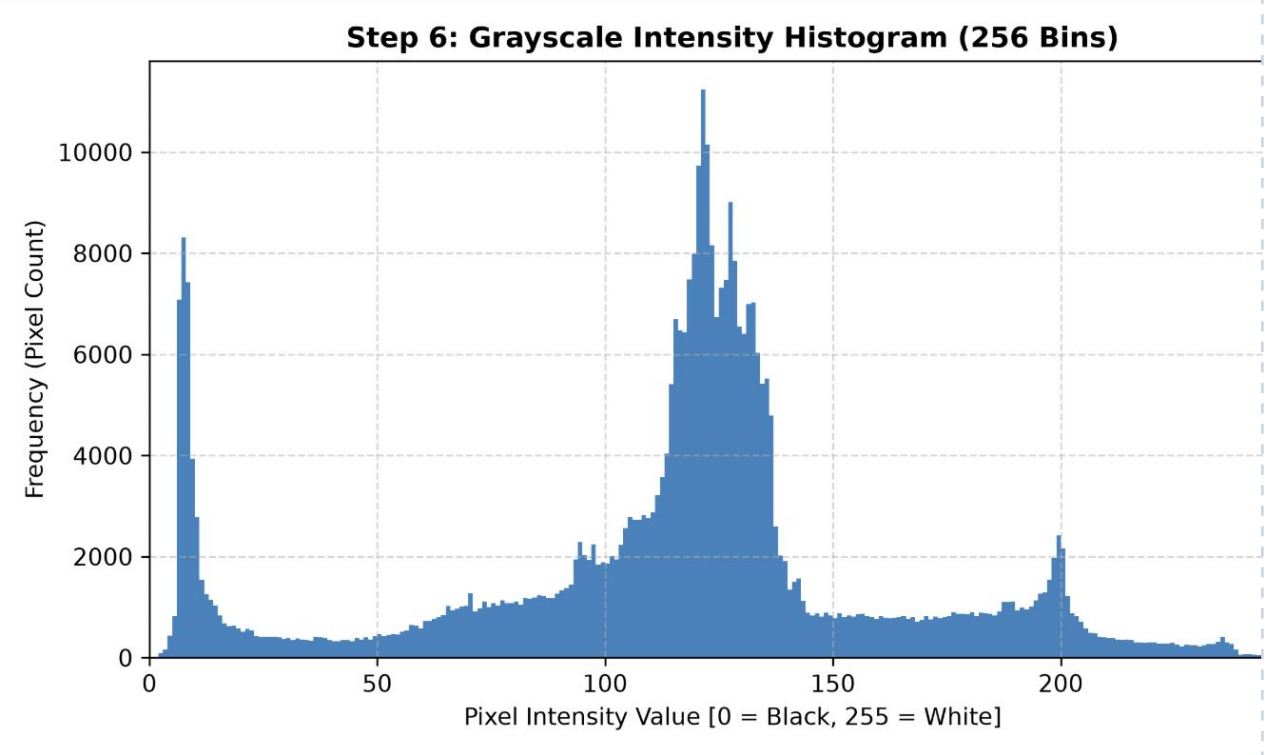
6. Step 6: Grayscale Histogram Analysis

Flattened the grayscale image and plotted a 256-bin histogram over range [0, 256]. Saved as 'grayscale_histogram.png'.

```
# ------------------------------------------------------------
# STEP 6: GRAYSCALE HISTOGRAM & ANALYSIS
# ------------------------------------------------------------
print("\n[STEP 6: GRAYSCALE HISTOGRAM]")
plt.figure(figsize=(8, 4.5))
plt.hist(
    img_gray.ravel(),
    bins=256,
    range=[0, 256],
    color="#2B6CB0",
    alpha=0.85,
    edgecolor="none",
)
plt.title(
    "Step 6: Grayscale Intensity Histogram (256 Bins)",
    fontsize=12,
```

```
    fontweight="bold",
)
plt.xlabel("Pixel Intensity Value [0 = Black, 255 = White]")
plt.ylabel("Frequency (Pixel Count)")
plt.xlim([0, 256])
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("grayscale_histogram.png", dpi=300)
plt.show()

mean_val = np.mean(img_gray)
std_val = np.std(img_gray)
print(f"  • Grayscale Mean Intensity: {mean_val:.2f}")
print(f"  • Grayscale Standard Deviation: {std_val:.2f}")
```



```
[STEP 6: GRAYSCALE HISTOGRAM]
  • Grayscale Mean Intensity: 113.64
  • Grayscale Standard Deviation: 50.90
```

Tonal Evaluation: The image has a well-balanced tonal distribution with a mean intensity of 113.64 and standard deviation of 50.90. The major peak occurs in the midtone range (bins 115–135) representing the background foliage and plumage, while secondary peaks near bin 10 capture the dark shadows and eye details without highlight clipping.

7. Step 7: Bit Depth Quantization

Generated an 8-bit synthetic gradient (0–255) and quantized to 4-bit (16 levels), 2-bit (4 levels), and 1-bit (2 levels). Verified unique gray levels and visualized false contouring.

```
# ------------------------------------------------------------
  # STEP 7: BIT DEPTH QUANTIZATION
  # ------------------------------------------------------------
  print("\n[STEP 7: BIT DEPTH DEMONSTRATION]")
  gradient_row = np.linspace(0, 255, 512, dtype=np.uint8)
  grad_8bit = np.tile(gradient_row, (120, 1))

  # Quantization to 1-bit, 2-bit, and 4-bit
  grad_1bit = (grad_8bit // 128) * 255
  grad_2bit = (grad_8bit // 64) * 85
  grad_4bit = (grad_8bit // 16) * 17
```

```
u_8bit = len(np.unique(grad_8bit))
u_4bit = len(np.unique(grad_4bit))
u_2bit = len(np.unique(grad_2bit))
u_1bit = len(np.unique(grad_1bit))
```

```
print(f"  • 8-bit Unique Levels: {u_8bit} (Expected: 256)")
print(f"  • 4-bit Unique Levels: {u_4bit} (Expected: 16)")
print(f"  • 2-bit Unique Levels: {u_2bit} (Expected: 4)")
print(f"  • 1-bit Unique Levels: {u_1bit} (Expected: 2)")
```

```
fig, axes = plt.subplots(4, 1, figsize=(10, 7))
axes[0].imshow(grad_8bit, cmap="gray")
axes[0].set_title(f"8-bit Gradient ({u_8bit} unique levels)")
axes[0].axis("off")
```

```
axes[1].imshow(grad_4bit, cmap="gray")
axes[1].set_title(f"4-bit Quantized Gradient ({u_4bit} unique levels)")
axes[1].axis("off")
```

```
axes[2].imshow(grad_2bit, cmap="gray")
axes[2].set_title(f"2-bit Quantized Gradient ({u_2bit} unique levels)")
axes[2].axis("off")
```

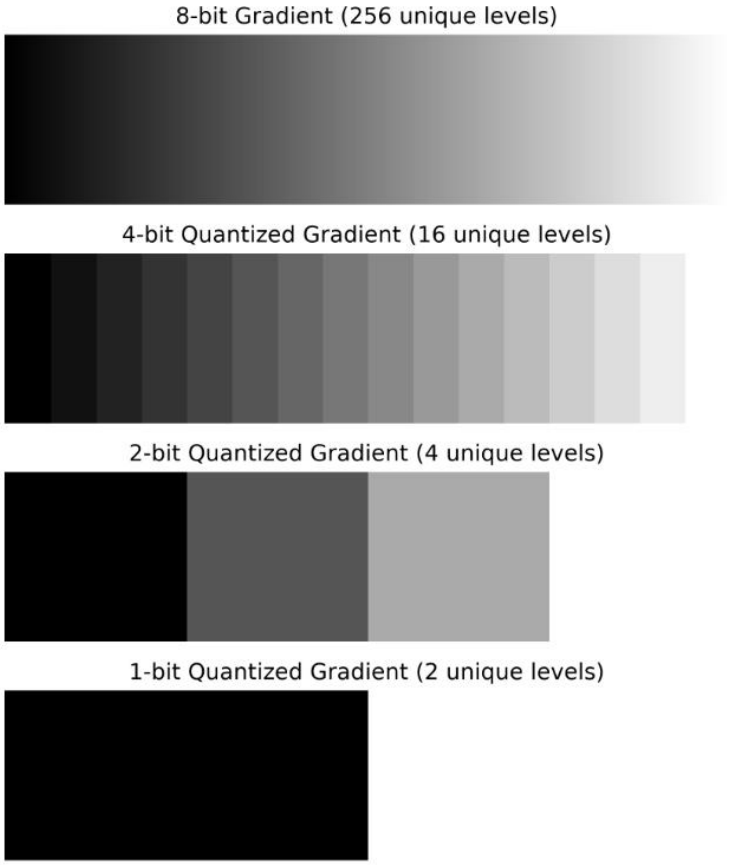
```
axes[3].imshow(grad_1bit, cmap="gray")
axes[3].set_title(f"1-bit Quantized Gradient ({u_1bit} unique levels)")
axes[3].axis("off")
```

```
plt.suptitle(
    "Step 7: Amplitude Quantization Across Varying Bit Depths",
    fontsize=13,
    fontweight="bold",
```

```
)
plt.tight_layout()
plt.savefig("bit_depth_comparison.png", dpi=300)
plt.show()
```

```
print("\n" + "=" * 65)
print("  ALL LAB 02 TASKS COMPLETED SUCCESSFULLY!  ")
print("=" * 65)
```

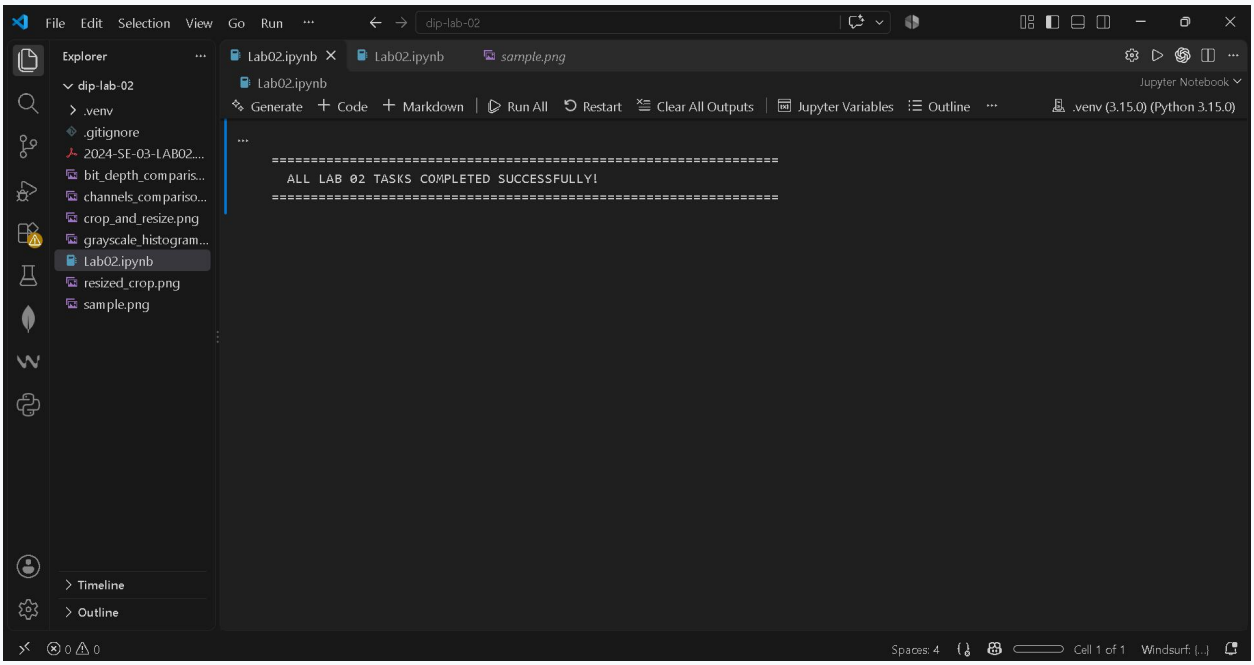
Step 7: Amplitude Quantization Across Varying Bit Depths



```
[STEP 7: BIT DEPTH DEMONSTRATION]
  • 8-bit Unique Levels: 256 (Expected: 256)
  • 4-bit Unique Levels: 16 (Expected: 16)
  • 2-bit Unique Levels: 4 (Expected: 4)
  • 1-bit Unique Levels: 2 (Expected: 2)
```

8. Step 8: Full Execution Verification

The Jupyter notebook lab02.ipynb was executed cell-by-cell sequentially within the active (.venv) kernel. All outputs, plots, and files were successfully generated.



9. Critical Conceptual Questions

9.1. BGR vs. RGB Color Representations

• Difference: BGR and RGB store the identical color data, differing only in byte channel ordering (Blue-Green-Red vs. Red-Green-Blue).

• Origin: OpenCV uses BGR due to legacy Windows bitmap (BMP) and early hardware frame-grabber byte formats.

• Display Effect: Matplotlib expects RGB. If an unconverted BGR image is passed directly to plt.imshow(), the red and blue channels swap, causing red objects and skin tones to turn alien blue, and blue objects to turn brownish-orange.

9.2. Spatial Resolution vs. Intensity Bit Depth

• Spatial Resolution (Sampling): The number of pixels across physical space (Width × Height / DPI). Governs geometric detail and fine structures. Decreasing it causes pixelation and blocky stair-stepping (aliasing).

• Bit Depth (Quantization): The number of bits per pixel determining discrete gray/color levels (2^b). Governs tonal smoothness and color gradation. Decreasing it causes false contouring (posterization/banding).

9.3. What an Image Histogram Shows vs. What It Does Not Show

• What it Shows: Global tonal distribution, dynamic range, overall contrast, mean exposure level, and clipping in shadows (bin 0) or highlights (bin 255).

• What it Does NOT Show: Zero spatial information. It completely ignores where pixels are physically located. A photo of a bird, a checkerboard, or randomized noise with the same pixel counts will yield identical histograms. It cannot detect shapes, textures, or geometry.